



Name:- JayKishan J Saharan

Div:- D Roll no.:- 26

Course/Program:- SY BTech CSE

Subject:- Data Structure and Algorithm.

~~Q.1~~ Theory assignment 2

Q.1] Write an algorithm for binary search and explain its working with appropriate example passwise?

Soln:-

Binary Search is the search technique that works efficiently on sorted lists.

Binary Search follows the divide and Conquer approach in which the list is divided into two halves, and the item is compared with the middle element of the list.

If the match is found then the location of the middle element is returned.

otherwise, we search into either of the halves depending upon the result produced through the match.

Algorithm - Binary Search (a, lower-bound, upper-bound, val).

Step. 1 beg = lower-bound, end = upper-bound, pos = -1

Step. 2 Repeat Step 3 & 4 while beg <= end

Step. 3 mid = (beg + end) / 2

Step. 4 if a[mid] == val

pos = mid

Print pos

Go to Step 6

else if a[mid] > val

end = mid - 1

else

beg = mid + 1

Step: 5 if pos = -1

Print "value is not present in the array"

Step: 6 exit.

Example:- value present in array.

let the sorted array be:

$a = [5, 15, 25, 35, 45, 55, 65]$

lower-bound = 0

upper-bound = 6

val = 45

Pass 1

- beg = 0, end = 6
- $mid = (0+6)/2 = 3$
- $a[3] = 35$
- Since array $\rightarrow$ set $big = mid + 1 = 4$.

Pass 2

- beg = 4, end = 6
- $mid = (4+6)/2 = 5$
- $a[5] = 55$
- Since $55 > 45 \rightarrow$ set $end = mid - 1 = 4$.

Pass 3

- beg = 4, end = 4
- $mid = (4+4)/2 = 4$
- $a[4] = 45 = val$
- found $\rightarrow$ pos = 4.

output :- value found at position 4.



Q.2] Compare Linear Search and Binary Search.

→

Linear Search

1. Works on unsorted or sorted array.
2. checks elements one by one
3. Time complexity (worst case) = $O(n)$
4. Simple and easy to implement.
5. Efficient for small datasets
6. No precondition (array can be random)
7. Best case : $O(1)$ (if element is 1st)
8. Space complexity = $O(1)$.
9. Example: Searching a name in an unordered attendance list.

Binary Search.

1. Works only on sorted arrays.
2. Divides the array into halves each step.
3. Time complexity (worst case) = $O(\log n)$
4. Slightly more complex than linear
5. Efficient for large datasets.
6. Requires data to be sorted.
7. Best case : $O(1)$ (if element is mid)
8. Space complexity = $O(1)$ (iterative).
9. Example: Searching a word in a dictionary.

Q.3] Write an algorithm / Pseudocode for selection sort. explain it's working with example.

Solⁿ:- Selection sort is another simple sorting algorithm that works by repeatedly selecting the minimum element from the unsorted portion of the array and moving it to beginning it divides the array into two parts



the sorted part and the unsorted part.

Algorithm.

- Step: 1 start with the first element as the minimum.
- Step: 2 Iterate through the unsorted portion of the list to find the minimum element.
- Step: 3 Swap the minimum element found with the first element of the unsorted portion.
- Step: 4 Move the boundary between sorted and unsorted portion one element to the right.
- Step: 5 Repeat until the entire list is sorted.

PseudoCode Selection Sort.

function SelectionSort (array):

$n = \text{length}(\text{array})$

 for i from 0 to $n-1$;

$\text{minIndex} = i$

 for j from $i+1$ to $n-1$;

 if $\text{array}[j] < \text{array}[\text{minIndex}]$:

$\text{minIndex} = j$

 Swap ($\text{array}[i]$, $\text{array}[\text{minIndex}]$)

 return array

Let's Sort the array $[64, 25, 12, 22, 11]$ using Selection Sort.

1. Initial Array = $[64, 25, 12, 22, 11]$

• $i = 0$: Minimum is 64 (index 0)

• Compare with 25, 12, 22, 11 $\rightarrow$ found 11 (index 4)

• Swap: $[11, 25, 12, 22, 64]$



2. Next iteration: $[11, 25, 12, 22, 64]$

- $i=1$: Minimum is 25 (index 1)
- compare with 12, 22, 64 $\rightarrow$ found 12 (index 2)
- Swap: $[11, 12, 25, 22, 64]$

3. Next iteration: $[11, 12, 25, 22, 64]$

- $i=1$: Minimum is 25 (index 1)
- compare with 22, 64 $\rightarrow$ found 22 (index 3)
- Swap: $[11, 12, 22, 25, 64]$

4. Next iteration: $[11, 12, 22, 25, 64]$

- $i=3$: Minimum is 25 (index 3)
- compare with 64 $\rightarrow$ no change
- Swap: $[11, 12, 22, 25, 64]$ (No change)

5. Last element: $[11, 12, 22, 25, 64]$

- $i=4$: Already Sorted.

Final Sorted Array is $[11, 12, 22, 25, 64]$

Summary:

- Time complexity: $O(n^2)$
- Space complexity: $O(1)$ (in-place)
- Stable-No.

Q.4) Write an algorithm / Pseudocode for Insertion sort and explain it's working with Example?

Solⁿ:- Bubble sort is a popular sorting algorithm that we prefer to sort the components of an array in a clear and particular order. Essentially in this algorithm, we need to compare the two adjacent element and perform a swap operation on them until they do not come in the



planned order. If the positions of the Comparators are right, then we have to move to the next iteration else, we need to perform a swap strategy.

How Bubble Sort works.

Step-by-Step process:

- Start at the beginning of the list.
- Compare the first two elements.
- Swap them if they are in the wrong order.
- Move to the next pair and repeat.
- Repeat the process until no swap are needed.

Pseudo code for bubble sort:

Initialize $n = \text{length of array}$.

Bubblesort (Array, n)

```
{
  for  $i = 0$  to  $n - 2$ 
  {
    for  $j = 0$  to  $n - 2$ 
    {
      if  $\text{Array}[j] > \text{array}[j+1]$ 
      swap ( $\text{Array}[j], \text{Array}[j+1]$ )
    }
  }
}
```

- Time complexity: Best time complexity: $O(n)$
Average time complexity: $O(n^2)$
Worst time complexity: $O(n^2)$



Q.5) Write an algorithm/Pseudocode for insertion sort and explain its working with example?

Solⁿ:-

Insertion sort is a simple and intuitive sorting algorithm that builds the final sorted array (or list) one item at a time. It is much like sorting playing cards in your hands. The algorithm is efficient for small data sets and has a time complexity of $O(n^2)$ $O(n^2)$ $O(n^2)$ in the average and worst cases, but it performs better than Selection Sort on avg.

Algorithm.

- Step: 1 Start from the second element (index 1) of the array.
- Step: 2 Compare the current elements with the elements in the sorted position (to the left).
- Step: 3 Shift the larger elements in the sorted position to the right to make space for the current element.
- Step: 4 Insert the current element into its current position.
- Step: 5 Repeat until the entire array is sorted.

Pseudocode for Insertion Sort:

function InsertionSort (array):

n = length (array)

for i from 1 to n-1:

Key = array[i]

j = i - 1

while j >= 0 and array[j] > Key:

array[j+1] = array[j]

j = j - 1

array[j+1] = Key
return array



Example:

let sort the array $[64, 25, 12, 22, 11]$ using Insertion Sort.

1. Initial array: $[64, 25, 12, 22, 11]$

- $i=1$: Key = 25
- compare 25 with 64: Shift 64 $\rightarrow [64, 64, 12, 22, 11]$
- Insert 25: $[25, 64, 12, 22, 11]$

2. Next iteration: $[25, 64, 12, 22, 11]$

- $i=2$: Key = 12
- compare 12 with 64: Shift 64 $\rightarrow [25, 25, 64, 22, 11]$
- compare 12 with 25: Shift 25 $\rightarrow [25, 25, 64, 22, 11]$
- Insert 12: $[12, 25, 64, 22, 11]$

3. Next iteration: $[12, 25, 64, 22, 11]$

- $i=3$: Key = 22
- compare 22 with 64: Shift 64 $\rightarrow [12, 25, 64, 64, 11]$
- compare 22 with 25: Shift 25 $\rightarrow [12, 25, 25, 64, 11]$
- compare 22 with 12: No shift $\rightarrow [12, 22, 25, 64, 11]$
- Insert 22: $[12, 22, 25, 64, 11]$

4. Next iteration: $[12, 22, 25, 64, 11]$

- $i=4$: Key = 11
- compare 11 with 64: Shift 64 $\rightarrow [12, 22, 25, 64, 64]$
- compare 11 with 25: Shift 25 $\rightarrow [12, 22, 25, 25, 64]$
- compare 11 with 22: Shift 22 $\rightarrow [12, 22, 22, 25, 64]$
- compare 11 with 12: Shift 12 $\rightarrow [12, 12, 22, 25, 64]$
- Insert 11: $[11, 12, 22, 25, 64]$

Final sorted array is: $[11, 12, 22, 25, 64]$.



Summary:

- Time complexity: Best case: $O(n)$ {when array is already sort}
Avg/worst case: $O(n^2)$
- Space complexity - $O(1)$ (in-place)
- Stable - Yes.

Q.6) Write an algorithm/pseudocode for Quicksort and apply Quicksort algorithm for sorting the following data set Stepwise [25, 82, 17, 23, 38, 7, 64, 86, 21].

Solⁿ:

Quick Sort is a highly efficient Sorting algorithm and is based on the divide-and-conquer principle. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays: those less than the pivot and those greater than the pivot. The sub-arrays are then sorted respectively.

Algorithm:

- Step: 1 choose a pivot: Selecting an element from the array as the pivot. This can be done in various ways (eg., first element, last element, random element, median)
- Step: 2 partitioning: Rearrange the array so that all elements less than the pivot are on the left, and all elements greater than the pivot are on the right. The pivot will be in its first position.
- Step: 3 Recursion: Recursively apply the above steps to the sub-arrays formed on the left and right of the pivot.



Pseudocode Quick Sort:

function quicksort (array, low, high):

if low < high

 pivot index = partition (array, low, high)

 quicksort (array, low, pivot index - 1) // Recursively

 Sort the left sub-array.

 quicksort (array, pivot index + 1, high) // Recursively sort
 the right sub array.

function partition (array, low, high):

 pivot = array [high] // choosing the last element as the pivot.

 i = low - 1 // Pointer for the small element.

 for j from low to high - 1 :

 if array [j] < pivot:

 i = i + 1

 swap (array [i], array [high])

 swap (array [i + 1], array [high])

 return i + 1

→ Data: 25, 82, 17, 23, 38, 7, 64, 86, 21.

Step: 1 choose pivot = 25

partition array → element smaller left, bigger right.

[17, 23, 7, 21] 25 [82, 38, 64, 86]

Step: 2 Sort left part, [17, 23, 7, 21] (Pivot = 17)

Partition:

[7] 17 [23, 21]

Sort [23, 21] (Pivot = 23):

[21] 23

So left part = [7, 17, 21, 23]



Step: 3 Sort right part [82, 38, 64, 86] (Pivot = 82)

Partition:

[38, 64] 82 [86]

Sort [38, 64] (Pivot = 38):

[38, 64]

So right part = [38, 64, 82, 86]

Final Sorted array:

[7, 17, 21, 23, 25, 38, 64, 82, 86]

Q.7) Write an algorithm Pseudocode for Merge Sort and apply, Merge Sort algorithm for sorting the following dataset stepwise [18, 12, 12, 22, 15, 24, 10, 16, 19, 14, 30]

Solⁿ:- Merge Sort is a classic divide and conquer sorting algorithm. It works by recursively dividing the array into halves until each sub array contains a single element and then merging those sub-arrays back together to form a sorted array.

Algorithm:

Step: 1 Divide: Split the array into two halves.

Step: 2 Conquer: Recursively sort each half

Step: 3 Merge: Combine the two sorted halves into a single sorted array.

Pseudocode for merge sort.

function mergeSort (array):

if length (array) ≤ 1 :

return array



```
mid = length(array) / 2  
left = mergesort(array[0:mid])  
right = mergesort(array[mid:length(array)])  
return merge(left + right)
```

for

function merge(left + right):

Sorted array = []

i = 0

j = 0

while i < length(left) and j < length(right):

if left[i] < right[j]:

append left[i] to Sorted array

i = i + 1

else:

append right[j] to Sorted Array.

j = j + 1

while i < length(left):

append left[i] to Sorted Array.

i = i + 1

while j < length(right):

append right[j] to sorted array.

j = j + 1

return Sorted array.

Data:

18, 13, 12, 22, 15, 24, 10, 16, 19, 14, 30.

Step: 1 & Divide the array into two halves.

[18, 13, 12, 22, 15] [24, 10, 16, 19, 14, 30]



Step 2: Step 2: Divide further

left side: $\rightarrow [18, 13, 12, 22, 15]$

$[18, 13] [12, 22, 15]$

Right side $\rightarrow [24, 10, 16, 19, 14, 30]$

$[24, 10, 16] [19, 14, 30]$

Step: 3

Keep dividing until single elements.

$[18] [13] [12] [22] [15] [24] [10] [16] [19] [14] [30]$

Step 4:

Merge Step by step (Sorting while merging)

• Merge $[18] [13] \rightarrow [13, 18]$

• Merge $[12] [22] \rightarrow [12, 22] \rightarrow$ merge with $[15] \rightarrow [12, 15, 22]$

• Merge $[24] [10] \rightarrow [10, 24] \rightarrow$ merge with $[16] \rightarrow [10, 16, 24]$

• Merge $[19] [14] \rightarrow [14, 19] \rightarrow$ merge with $[30] \rightarrow [14, 19, 30]$

Step 5:

Merge bigger parts

• Left side $[13, 18] + [12, 15, 22] \rightarrow [12, 13, 15, 18, 22]$

• Right side $[10, 16, 24] + [14, 19, 30] \rightarrow [10, 14, 16, 19, 24, 30]$

6:

final merge:

Merge $[12, 13, 15, 18, 22] + [10, 14, 16, 19, 24, 30]$

Result $[10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30]$

Q.8)

Sort 18, 13, 12, 22, 15, 24, 10, 16, 19, 14, 30 by using Bubble, Selection and insertion sort.

Solⁿ:-

1. Bubble Sort.

Bubble Sort repeatedly swap adjacent elements if they are in the wrong order.



Pass 1:

13, 12, 18, 15, 22, 10, 16, 19, 14, 24, 30

Pass 2:

12, 13, 15, 18, 10, 16, 19, 14, 22, 24, 30

Pass 3:

12, 13, 15, 10, 16, 18, 14, 19, 22, 24, 30

Pass 4:

12, 13, 10, 15, 16, 14, 18, 19, 22, 24, 30

Pass 5:-

12, 10, 13, 15, 14, 16, 18, 19, 22, 24, 30

Pass 6:-

10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30

Sorted Bubble:-

10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30

2. Selection Sort:

Selection Sort repeatedly selects the smallest element and puts in the correct place.

Pass 1: Smallest = 10 → Swap with 18.

10, 13, 12, 22, 15, 24, 18, 16, 19, 14, 30



Pass 2: Next Smallest = 12 $\rightarrow$ Swap with 13.

10, 12, 13, 22, 15, 24, 18, 16, 19, 14, 30.

Pass 3: Next Smallest = 13 $\rightarrow$ already in place

Pass 4: Next Smallest = 12 $\rightarrow$ Swap with 13.

10, 12, 13, 22, 15, 24, 18, 16, 19, 14, 30.

Pass 5: Next Smallest = 13 $\rightarrow$ already in place.

Pass 6: Next Smallest = 14 $\rightarrow$ Swap with 22.

10, 12, 13, 14, 15, 16, 18, 24, 19, 22, 30.

Pass 7: Next Smallest = 18 $\rightarrow$ already in place.

Pass 8: Next Smallest = 19 $\rightarrow$ Swap with 24.

10, 12, 13, 14, 15, 16, 18, 19, 24, 22, 30.

Pass 9: Next Smallest = 22 $\rightarrow$ Swap with 24

10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30.

Sorted (Selection):

[10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30]

3. Insertion Sort:

Insertion Sort builds the sorted array one element at a time.

Start [18] (Sorted), rest unsorted:



Pass 1: Insert 13 $\rightarrow$ [13, 18]

Pass 2: Insert 12 $\rightarrow$ [12, 13, 18]

Pass 3: Insert 22 $\rightarrow$ [12, 13, 18, 22]

Pass 4: Insert 15 $\rightarrow$ [12, 13, 15, 18, 22]

Pass 5: Insert 24 $\rightarrow$ [12, 13, 15, 18, 22, 24]

Pass 6: Insert 10 $\rightarrow$ [10, 12, 13, 15, 18, 22, 24]

Pass 7: Insert 16 $\rightarrow$ [10, 12, 13, 15, 16, 18, 22, 24]

Pass 8: Insert 19 $\rightarrow$ [10, 12, 13, 15, 16, 18, 19, 22, 24]

Pass 9: Insert 14 $\rightarrow$ [10, 12, 13, 14, 15, 16, 18, 19, 22, 24]

Pass 10: Insert 30 $\rightarrow$ [10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30]

Sorted (Insertion):

[10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30]

Final ans: All sort's gives same result.

10, 12, 13, 14, 15, 16, 18, 19, 22, 24, 30.